
Slice Engine Specification

Confinement

Confinement is provided by limiting the awareness of each other's cells (and links) to progressively smaller 'knowledge' with an increasing number of hops: (0) Zero Hops: Self - I know everything about myself. (1) One Hop: I know the identity of my neighbor, the liveness of her cell agent. (2) Two Hops. I do not know the identity of cells two hops away, but I do know the status of the topology (2 bits: live/not live). (3) Three Hops. I know only of the existence (1 bit: connected/disconnected).

Cells vs. Ports, vs. Links

A hypercell (unit of reliable computation) is a 3×3 tile (a self-cell with 8 ports, connecting to 8 neighbor cells). This is a hypercell with a unique identifier – where each cell shares its Unique identifier with its 8 neighbors, but not beyond. Tiles overlap such that every cell sees itself as the center of its own 'relative addressing' world, confined to '1-hop' out with 'identity awareness'. Enough to create a fault-tolerant unit.

Two hops out gives us temporal awareness (but not identity awareness) for $5 \times 5 = 25$ Cells ($8 \times 25 = 200$ link views). We could no longer fit this in the remaining 256 bits of the 64 byte frame, but we could extend it to a 128 byte frame. Although the number of links that can be monitored at this level (slices 2+3 is 64 – this includes many links 'seen' from both sides, so the actual number of cells monitored is only 25).

Three hops out, with 1 bit of status per link would (if we tried to extend to that) would be $7 \times 7 = 49$ Cells ($8 \times 49 = 392$ link views). It is at this point that the geometric scaling of the information in the mesh network grows past the point where (a) it is useful for connection awareness and tree healing, and (b) it is undesirable to have this much traffic on the links when they are otherwise basically idle.

Arrows

There are several different kinds of arrows, which represent some notion of 'direction' in the protocol. The slice engine, and higher layer protocols includes places where these can be specified as tools:

BIPARTITE LINK On initialization, each link has a way to 'break symmetry', so each port can distinguish itself from its partner on the same link. This status (POLARITY) is remembered on both sides of the link, and included in *every* packet slice ONE (Link liveness), and slice TWO, (Cluster Liveness).

SERIALIZABLE PETRIs Petri Tokens control the serializability for for Alice's and Bob's computations. These tokens use to the Stop and Wait (SAW) protocol to constantly alternate their 'causality': first allowing Alice to forward *her* program, and then allowing Bob to progress her token *on her behalf*. The same goes for Bob's token: first allowing Bob to forward *his* program, and then Bob allowing Alice to forward his program. The two Petri tokens represent to dual (competing) parallel executions.

DAG DIRECTION Above the Liveness Tensor layer, we build spanning Trees on which we can stack application graphs. The first step is for each cell to build a DAG rooted on itself. Each of these trees will have a corresponding DAG Direction.

APPLICATION QUIVERS Applications build causality structures, which comprise a 'substrate' on which they can organize flows. For example, Load Balancers, Erasure Coded Storage Shards, Database transaction schedules. These are described in more detail in the application graph document 'quivers.pdf'.

Objects and Relationships

What we have learned from Quantum mechanics (the study of what things are doing when you are not watching them) is that the *relationships* are just as (or maybe even more) real than the objects themselves. Our focus in this specification is therefore not on the objects (the cells – encapsulated units of compute, memory storage & packet processing), but on the connections between them. Mathematically, we refer to them as [morphisms](#), and the mathematical tools are provided by category theory (see document 'transformer.pdf').

Alice and Bob cannot know (the 'ontology') of what their packets are doing once they have sent them. They are constrained *by nature to waiting anxiously* for the response to that packet – which may never come – because networks may drop those packets, and we may never know if they were dropped on the way to the

destination node, on the way back from the destination node, or if it was the node itself (and the agents on them) which failed to process or return a response to the packet.

This is a deep technical issue addressed in the document [Dual-SAW-Spekkens.pdf](#). For our purposes here, we need to be mathematically precise and distinguish between objects (the cell agents) and the relationships (the morphisms). This is the problem that this specification addresses directly.

What Alice can see is the epistemic fragments of ‘knowledge’ that are left behind as the last packet ‘I’ sent, and the last packet ‘I’ received. It is impossible to ‘know’ or keep a memory of everything that happened to the packet on the way from one node to another. This is a well-known problem in quantum theory, which is solved by precisely specifying the knowledge balance’ relationships between the epistemic fragments on both sides (Alice’s view and Bob’s view).

Slice Engine Hardware

The slice engine is designed to exploit the 64-bit (8 byte) flit size of typical FPGAs’ SerDes. 64 bits can be parallelized and processed in one clock cycle, with its result available to the logic in the next clock cycle (providing timing is met).

We distinguish between the [Flit](#) size and [stop and wait](#) (SAW) size. Flits are modular units are protocol bits, processed together from the stream. Header flits, body flits and tail flits are forwarded through the network whether or not the destination is ready. A SAW protocol on the other hand does not send another flit unless it has processed the previous one. This is the opposite (promise theory) polarity of PAUSE frames, which imply “keep on sending until I tell you to stop”.

SAW protocols will immediately be rejected by veteran protocol designers as wasteful and not worth further consideration. The following are typical objections:

1. The protocol will freeze if acknowledgments are lost.
2. The bandwidth “on the wire” will be wasted unless multiple packets (flits) are in flight at any one time.
3. The bandwidth “in the buffers” will be wasted unless the protocol can get the stream started and start moving packets to the network.

Those objections are from the perspective of an architecture that separates the infrastructure into servers (nodes) and switches/routers (for example an inter-connect infrastructure which is designed, sold, managed and upgraded by a team of engineers and operations people separately from the server infrastructure, CPUs, memory, storage and operating systems).

When servers are connected directly, as they are in Tesla’s dojo, and Google’s Aquila,¹ then a different perspective comes into view:

- First the switch (TOR) and SmartNIC are merged into one unit that can act autonomously(Google Aquila perspective).
- Second. There is no need for intermediate transit stations to carry or switch packets because cells have only a single (pair) of transit/receive channels with no active components.
- Third. Each “Pair” of cells can perform useful computations with the minimum possible latency. With short cables inside a rack even the speed of light delays (of 1 ns per foot) is minimized.
- Even for multiple Hop situations, were the SmartNIC must provide forwarding to reach other nodes, the local node is also a potential target and must also be included in the multicast destination set.

Bandwidth vs. Latency

We start with the assumption of a 25 Gb single-lane SerDES. The technology already exists for 56 Gb using PAM-4 for 50 Gbits 112 Gbits with modest phase encoding. The technology roadmap extends to [224 Gbit and beyond](#) with 5nm process technologies, so we are not concerned about bandwidth in the future.

25 Gb and eight byte flits (64 bits) can be supported with modest 400 MHz FPGA’s.²

¹The [Tesla 2D mesh](#) incorporates a tile to tile interconnect using a similar 112 Gb SerDES as Ethernet interconnect them

²It is possible but extremely expensive to get to 1 GHz so we ignore that for our first generation products

We also know that we can build pretty good transmission lines for 6 inches or 12 inch cables inexpensively. This is already done with NVME, and SATA and USB-C. However, we focus in our first product is the lowest possible latency not cost.

Port-Addressing, the Problem

Each cell from its *self* perspective, will label its ports in a preferred way. Even though there are 8 physical ports per cell, there are also one or more ‘internal’ ports to the cell agent, otherwise there would be no mechanism to include the ‘up-tree’ in the treecast. Fig. 1a below shows the method of port addressing using labels $P[1..8]$, and Fig. 1b shows port addressing using labels $P[0..7]$, common in the networking community.

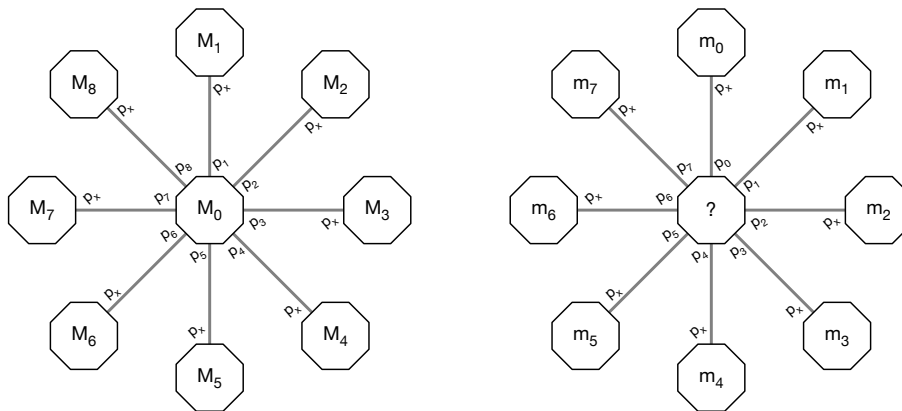


Figure 1: Port Addressing Problem

Problem: we have no (unique or relative) ‘identity’ for the self-cell, and the when packets come in on a port, we are unable to respond with the status of the cell above us.

Port-Addressing, the Solution

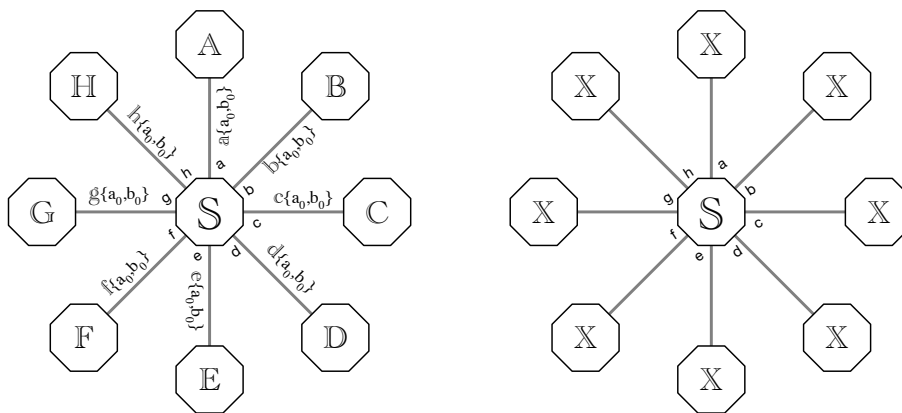
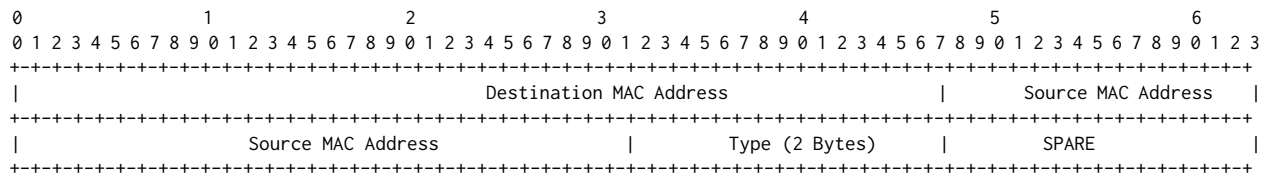
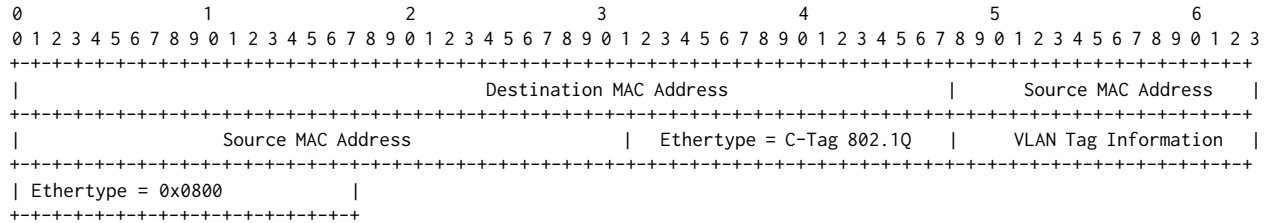


Figure 2: Port-Addressing – Proposed Solution

Packet Formats

[Q1] What does your packet format look like?

[A1] Compare the standard Ethernet Header (in 64-bit packet format):



[A1] With the Algo Fabrix Ethernet Header (64-bit packet format):

